

Bash Scripting - Deep Dive Handbook

1) What is a Bash script?

A Bash script is a plain-text file containing shell commands executed by `/bin/bash`. It automates tasks such as backups, deployments, data processing, user management, and monitoring.

```
#!/bin/bash
echo "Hello from bash"
ls -la
exit 0
```

Shebang

The first line `#!/bin/bash` selects the interpreter for this file. Other shells are possible, e.g. `#!/usr/bin/env bash` or `#!/bin/sh`.

2) Types of Bash scripts (by purpose)

- a) One-off utility scripts: a few lines to speed up repetitive tasks.
- b) System admin scripts: user, service, log rotation, backup, health checks.
- c) Data/ETL scripts: parse logs, transform CSVs, generate reports.
- d) Provisioning/CI scripts: build, test, deploy.
- e) Library-style scripts: define functions sourced by other scripts.

Typical structure

```
#!/usr/bin/env bash
set -Eeuo pipefail
IFS=$'\n\t'

log(){ printf "[%s] %s\n" "$(date +%T)" "$*"; }
main(){
    log "Starting"
    # your logic here
}
main "$@"
```

Best practice flags

```
set -e : exit on error
set -u : error on undefined vars
set -o pipefail : fail a pipeline if any stage fails
set -E : inherit ERR traps in functions
```

3) Variables in Bash - Detailed

Bash is dynamically typed: all variables are strings by default, but arithmetic contexts can treat them as integers. You can influence behavior with declare builtins and context.

3.1 Scalar variables (strings by default)

Bash Scripting - Deep Dive Handbook

```
name='Alice'
echo "Hello $name"
# No spaces around =, and no $ on assignment
unset name
```

3.2 Integers and arithmetic variables

Use arithmetic contexts or declare -i to get integer behavior.

```
count=10
((count++))          # arithmetic context
sum=$(( count + 5 ))
declare -i num=42
num+=8               # stays integer
echo "$sum $num"
```

3.3 Arrays (indexed)

```
declare -a nums=(10 20 30)
echo "First: ${nums[0]}"
echo "All:   ${nums[@]}"
echo "Idxs:  ${!nums[@]}"
echo "Len:   ${#nums[@]}"
nums+=(40 50)
unset 'nums[1]'      # remove element at index 1
```

3.4 Associative arrays (hash maps)

```
declare -A ages=( [alice]=30 [bob]=28 )
echo "Alice: ${ages[alice]}"
for k in "${!ages[@]}"; do echo "$k -> ${ages[$k]}"; done
```

3.5 Readonly, local, exported (environment)

```
readonly APP_HOME=/opt/app
export PATH="$HOME/bin:$PATH"
foo=global
myfunc(){ local foo=local; echo "$foo"; }
myfunc; echo "$foo" # local vs global
```

3.6 Positional and special parameters

```
script.sh arg1 arg2
# Inside script:
echo "$0" # script name
echo "$1" # first arg
echo "$#" # number of args
echo "$@" # all args (word-split)
echo "$*" # all args (single word with IFS)
echo "$?" # exit status of last command
echo "$$" # PID of current shell
echo "$!" # PID of last background job
```

3.7 Useful special variables

Bash Scripting - Deep Dive Handbook

```
echo "$RANDOM"    # random 0..32767
echo "$SECONDS"   # seconds since shell started
echo "$LINENO"    # current line number
echo "$BASH_VERSION"
```

3.8 Quoting rules (very important)

```
echo $var          # expanded, word-split, globbed
echo "$var"        # expanded, no word-split, no glob
echo '$var'        # no expansion
path="/tmp/a b"
rm "$path"         # always quote paths with spaces
```

3.9 Parameter expansion tricks

```
v=${v:-default}    # default if unset or empty
v=${v:=default}     # also assign default
: ${REQUIRED:?missing}
len=${#v}           # length
sub=${v:2:3}        # slice
x=${v/foo/bar}      # first substitution
x=${v//foo/bar}     # global substitution
x=${v#prefix}       # strip shortest prefix match
x=${v##prefix*}     # strip longest prefix match
x=${v%suffix}       # strip shortest suffix
x=${v%%suffix*}     # strip longest suffix
```

4) Operators in Bash - Types and Usage

4.1 Arithmetic operators (in (()) or \$(()))

```
(( a=5+3 ))      # + - * / % **
(( a+=2, b=a++ ))
result=$(( (a*b) % 7 ))
```

4.2 Numeric comparisons

In test/[] use -eq -ne -lt -le -gt -ge. In (()) you can use < > <= >= == !=.

```
a=10; b=20
if [ "$a" -lt "$b" ]; then echo lt; fi
if (( a < b )); then echo lt; fi
```

4.3 String operators

```
s="hello"
[ "$s" = "hello" ]
[ "$s" != "world" ]
[ -n "$s" ] # non-empty
[ -z "$t" ] # empty
# In [[ ]] you can also use < and > lexicographically
[[ "$s" < "zoo" ]] && echo "true"
```

4.4 Logical operators

Bash Scripting - Deep Dive Handbook

```
[ -f file ] && echo exists || echo missing
if [[ -d src && -x build.sh ]]; then ./build.sh; fi
! false && echo ok
```

4.5 File test operators

```
[ -e path ] # exists
[ -f file ] # regular file
[ -d dir ] # directory
[ -s file ] # size > 0
[ -r file ] # readable
[ -w file ] # writable
[ -x file ] # executable
[ -L link ] # symlink
[ file1 -nt file2 ] # newer than
[ file1 -ot file2 ] # older than
```

4.6 Regex matching (only in [[]])

```
s="abc123"
if [[ $s =~ ^[a-z]+[0-9]+$ ]]; then echo "match"; fi
```

Why use particular operators?

Use numeric operators inside (()) for arithmetic speed and clarity.

Use [[]] for safer string tests (no word-splitting, no pathname expansion).

Use file test operators to query filesystem state efficiently.

Pick the right operator family for the data type you are testing.

5) Conditional statements - purpose and usage

Conditionals choose different paths based on tests. In Bash you typically use if/elif/else or case/esac.

5.1 if / elif / else

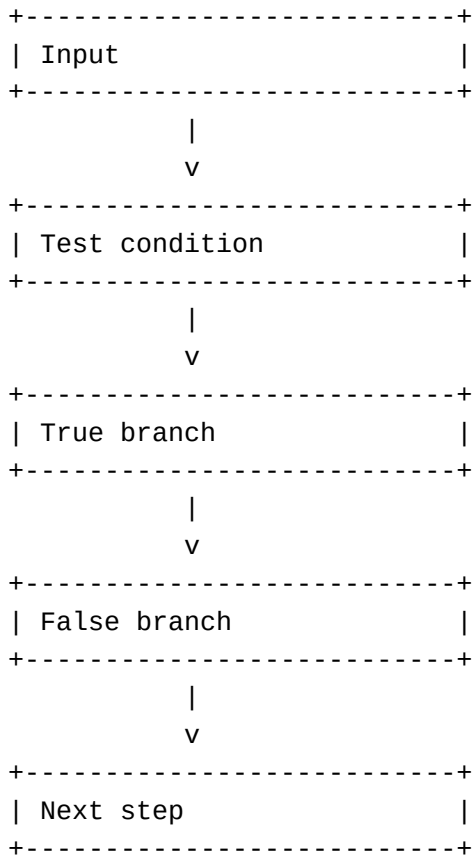
```
read -p "Age: " age
if (( age >= 18 )); then
    echo "Adult"
elif (( age >= 13 )); then
    echo "Teen"
else
    echo "Child"
fi
```

5.2 case / esac (pattern matching)

```
read -p "Answer [y/n]: " a
case "$a" in
    y|Y|yes) echo "Proceed";;
    n|N|no)  echo "Abort";;
    *)      echo "Invalid";;
esac
```

Diagram: conditional flow

Bash Scripting - Deep Dive Handbook



6) Loops - purpose, types, and definitions

Loops repeat a set of commands. Use them to process lists, files, or until a condition changes.

6.1 for-in over a list

```
for name in Alice Bob Carol; do
    echo "Hello $name"
done
```

6.2 C-style for loop

```
for ((i=0; i<5; i++)); do
    echo "$i"
done
```

6.3 while loop

```
i=1
while (( i <= 3 )); do
    echo "$i"; ((i++))
done
```

6.4 until loop

Bash Scripting - Deep Dive Handbook

```
x=0
until (( x == 3 )); do
    echo "$x"; ((x++))
done
```

6.5 break and continue

```
for i in {1..10}; do
    (( i==3 )) && continue
    (( i==6 )) && break
    echo "$i"
done
```

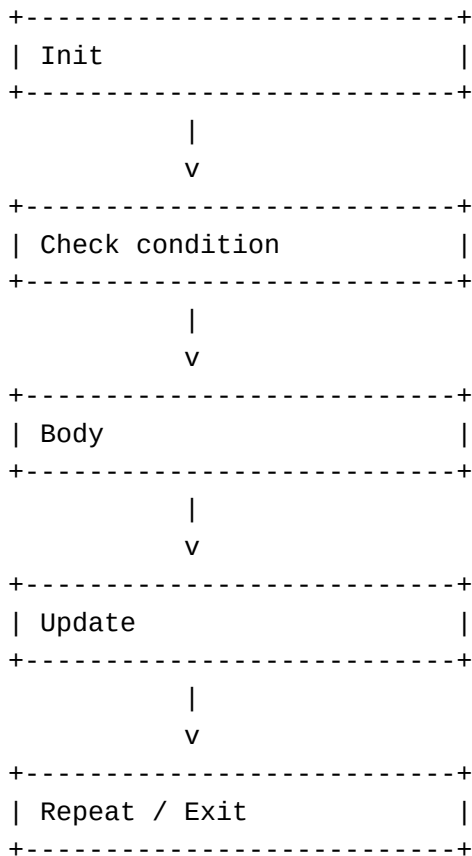
6.6 Reading files safely

```
while IFS= read -r line; do
    printf '%s\n' "$line"
done < input.txt
```

6.7 select loop (menus)

```
PS3='Pick a fruit: '
select f in apple banana cherry; do
    echo "You picked $f"; break
done
```

Diagram: loop flow



7) Functions, exit codes, and error handling

Bash Scripting - Deep Dive Handbook

```
add(){ echo $(( $1 + $2 )); }
result=$(add 2 3)
echo "$result" # 5

myjob(){
  cp source dest || return 1
}
if myjob; then echo OK; else echo FAIL; fi
```

Exit status

Every command returns an exit code in \$? . 0 means success; non-zero means failure.

Use 'set -e' to exit on error in simple scripts, but be mindful with commands that may legitimately fail.

8) I/O redirection and pipelines

```
cmd > out.txt           # stdout to file
cmd >> out.txt           # append
cmd 2> err.txt          # stderr to file
cmd > out 2>&1           # redirect both to out
cmd | other             # pipe stdout to other
cmd <<< "text"           # here-string
cat <<'EOF'
multiline content
EOF
```

9) Example: backup with logs and checks

Bash Scripting - Deep Dive Handbook

```
#!/usr/bin/env bash
set -Eeuo pipefail

SRC=${1:-$HOME/Documents}
DST=${2:-$HOME/Backup}
LOG=${3:-$HOME/backup.log}

ts(){ date '+%F %T'; }
log(){ printf '%s %s\n' "${ts}" "$*" | tee -a "$LOG"; }

log "Starting backup: $SRC -> $DST"
if [[ ! -d "$SRC" ]]; then
    log "Source missing: $SRC"; exit 1
fi
mkdir -p "$DST"
rsync -a --delete "$SRC"/ "$DST"/
rc=$?
if (( rc==0 )); then
    log "Backup completed successfully"
else
    log "Backup failed with code $rc"; exit $rc
fi
```